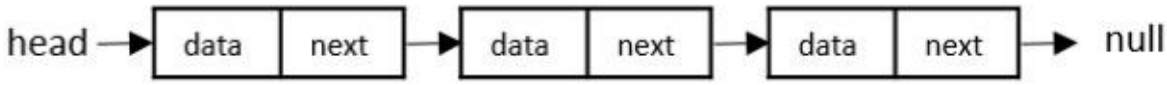
UNIT II

Singly linked lists: representation and operations, implementation of singly linked list. Doubly linked lists and circular linked lists, comparing arrays and linked lists, applications of linked lists.

A linked list is a linear data structure which can store a collection of "nodes" connected together via links i.e. pointers. Linked lists nodes are not stored at a contiguous location, rather they are linked using pointers to the different memory locations. A node consists of the data value and a pointer to the address of the next node within the linked list.

A linked list is a dynamic linear data structure whose memory size can be allocated or de-allocated at run time based on the operation insertion or deletion, this helps in using system memory efficiently. Linked lists can be used to implment various data structures like a stack, queue, graph, hash maps, etc.



A linked list starts with a **head** node which points to the first node. Every node consists of data which holds the actual data (value) associated with the node and a next pointer which holds the memory address of the next node in the linked list. The last node is called the tail node in the list which points to **null** indicating the end of the list.

Linked Lists vs Arrays

Arrays and linked lists are fundamental data structures in computer science, each with distinct characteristics that make them suitable for different scenarios.

Arrays are collections of elements stored in contiguous memory locations. This arrangement allows for constant-time (O(1)) random access to elements using their index. However, arrays have a fixed size, meaning their capacity must be defined at the time of creation and cannot be altered during runtime. Inserting or deleting elements, especially in the middle, can be inefficient because it may require shifting multiple elements to maintain order. Despite these drawbacks, arrays benefit from better cache locality due to their contiguous memory allocation, which can lead to performance improvements in certain scenarios.

Linked lists, on the other hand, consist of nodes where each node contains data and a reference (or link) to the next node in the sequence. This structure allows for dynamic memory allocation, meaning the list can grow or shrink at runtime as needed. Insertion and deletion operations are generally more efficient than arrays, as they involve updating pointers without the need for shifting elements. However, linked lists do not support efficient random access;

to access a specific element, one must traverse the list from the beginning, resulting in linear time complexity (O(n)) for such operations. Additionally, linked lists incur extra memory overhead due to the storage of pointers alongside data.

Key Differences:

- **Memory Allocation:** Arrays require contiguous memory, which can lead to memory allocation challenges, especially for large datasets. Linked lists, however, can be scattered throughout memory, as each node points to the next, allowing for more flexible memory utilization.
- **Size Flexibility:** Arrays have a fixed size determined at creation, making them less adaptable to dynamic data scenarios. Linked lists can adjust their size during runtime, offering greater flexibility for applications where the amount of data is unpredictable.
- **Performance Considerations:** Arrays provide faster access to elements due to their contiguous memory and support for random access. Linked lists, while offering efficient insertions and deletions, suffer from slower element access times due to their sequential nature.

When to Use:

- **Arrays** are preferable when the number of elements is known in advance and unlikely to change, and when fast access to elements is required.
- **Linked lists** are suitable for applications where the size of the dataset is dynamic, and frequent insertions and deletions occur, particularly in the middle of the structure.

Types of Linked List

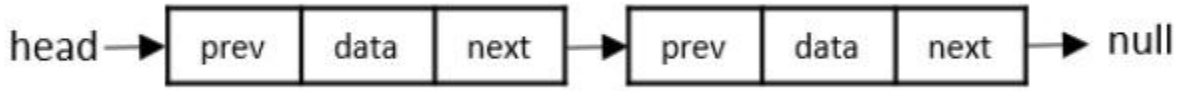
Following are the various types of linked list.

Singly Linked Lists

Singly linked lists contain two "buckets" in one node; one bucket holds the data and the other bucket holds the address of the next node of the list. Traversals can be done in one direction only as there is only a single link between two nodes of the same list.

Doubly Linked Lists

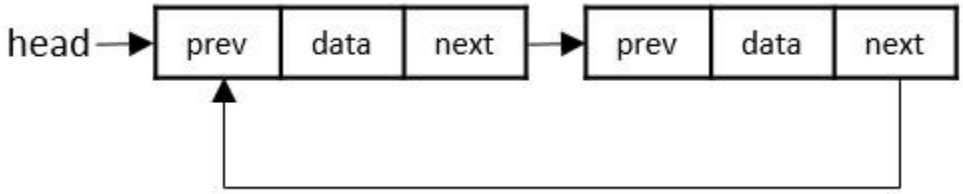
Doubly Linked Lists contain three "buckets" in one node; one bucket holds the data and the other buckets hold the addresses of the previous and next nodes in the list. The list is traversed twice as the nodes in the list are connected to each other from both sides.



Circular Linked Lists

Circular linked lists can exist in both singly linked list and doubly linked list.

Since the last node and the first node of the circular linked list are connected, the traversal in this linked list will go on forever until it is broken.

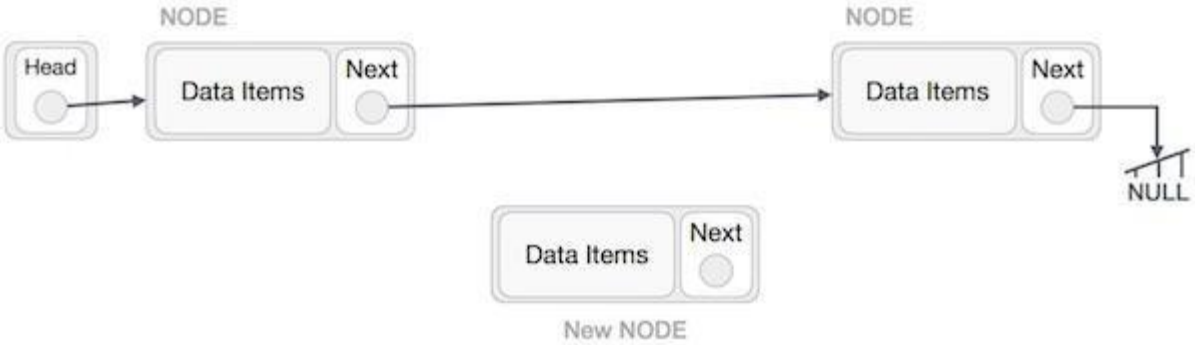


Basic Operations in Linked List

The basic operations in the linked lists are insertion, deletion, searching, display, and deleting an element at a given key. These operations are performed on Singly Linked Lists as given below –

- **Insertion** – Adds an element at the beginning of the list.
- **Deletion** – Deletes an element at the beginning of the list.
- **Display** – Displays the complete list.
- **Search** – Searches an element using the given key.
- **Delete** – Deletes an element using the given key.

Singly Linked List



Linked Lists in C (Without Double Pointers or Return Pointers)

Linked lists are dynamic data structures consisting of nodes connected by pointers. Unlike arrays, linked lists don't require contiguous memory; each node can be anywhere in memory and links to the next node in the sequence. This flexibility allows efficient insertion and deletion without reorganizing the entire data structure, though accessing an arbitrary element requires traversal from the head.

1. Singly Linked List

A **singly linked list** is the simplest form of a linked list. Each node contains two parts: data and a pointer to the next node. The last node's next pointer is set to NULL to indicate the end of the list. We maintain a global pointer head that points to the first node (or is NULL if the list is empty).

Node Representation

```
struct Node {
   int data;
   struct Node* next;
};

struct Node* head = NULL; // Global head pointer
```

Operations on a Singly Linked List

1. Insertion

- **At the Beginning**

```
void insertAtBeginning(int value) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->next = head;
   head = newNode;
}
```

- **At a Specific Position**

```
void insertAtPosition(int pos, int value) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
```

DATA SCIENCE

```
    if (pos == 1) {
        newNode->next = head;
        head = newNode;
        return;
    }
    struct Node* temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }
    if (temp == NULL) return;
    newNode->next = temp->next;
    temp->next = newNode;
}
```

- **At the End**

```
void insertAtEnd(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) {
        head = newNode;
        return;
    }
    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
```

DATA SCIENCE

2. Deletion

- **Delete from Beginning**

```
void deleteBeginning() {
   if (head == NULL) return;
   struct Node* temp = head;
   head = head->next;
   free(temp);
}
```

- **Delete from a Specific Position**

```
void deleteAtPosition(int pos) {
   if (head == NULL) return;
   if (pos == 1) {
      struct Node* temp = head;
      head = head->next;
      free(temp);
      return;
   }
    struct Node* temp = head;
   for (int i = 1; i < pos - 1 && temp->next != NULL; i++) {
      temp = temp->next;
   }
   if (temp->next == NULL) return;
  struct Node* toDelete = temp->next;
   temp->next = temp->next->next;
   free(toDelete);
}
```

- **Delete from End**

```
void deleteEnd() {
```

DATA SCIENCE

```
    if (head == NULL) return;
    if (head->next == NULL) {
        free(head);
        head = NULL;
        return;
    }

    struct Node* temp = head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}
```

3. Search

```
int search(int key) {
    struct Node* temp = head;
    while (temp != NULL) {
        if (temp->data == key) return 1;
        temp = temp->next;
    }
    return 0;
}
```

4. Display

```
void displayList() {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
```

DATA SCIENCE

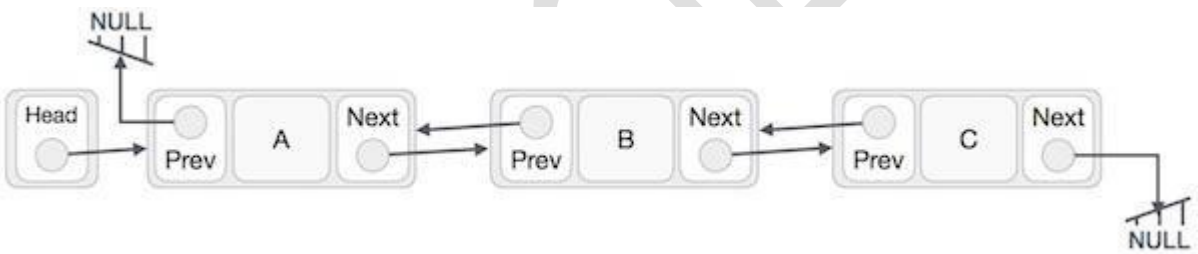
```
        temp = temp->next;
    }
    printf("NULL\n");
}
```

Doubly Linked List

A **Doubly Linked List (DLL)** consists of nodes where each node contains:

- **Data** (the value stored)
- **Next pointer** (address of the next node)
- **Previous pointer** (address of the previous node)

```
struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};
```



- Doubly Linked List contains a link element called first and last.
- Each link carries a data field(s) and a link field called next.
- Each link is linked with its next link using its next link.
- Each link is linked with its previous link using its previous link.
- The last link carries a link as null to mark the end of the list.

```
struct DNode {
    int data;
    struct DNode* next;
    struct DNode* prev;
};

struct DNode* head = NULL; // Global head pointer for DLL
```

Operations on a Doubly Linked List

1. Insertion

- **At the Beginning**

```
void insertAtBeginningDLL(int value) {
    struct DNode* newNode = (struct DNode*)malloc(sizeof(struct DNode));
    newNode->data = value;
```

```
    newNode->next = head;
    newNode->prev = NULL;
    if (head != NULL)
        head->prev = newNode;
    head = newNode;
}
```

- **At a Specific Position**

```
void insertAtPositionDLL(int pos, int value) {
    struct DNode* newNode = (struct DNode*)malloc(sizeof(struct DNode));
    newNode->data = value;

    if (pos == 1) {
        newNode->next = head;
        newNode->prev = NULL;
        if (head != NULL)
            head->prev = newNode;
        head = newNode;
        return;
    }

    struct DNode* temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }
    if (temp == NULL) return;

    newNode->next = temp->next;
    newNode->prev = temp;
    if (temp->next != NULL)
        temp->next->prev = newNode;
    temp->next = newNode;
}
```

- **At the End**

```
void insertAtEndDLL(int value) {
    struct DNode* newNode = (struct DNode*)malloc(sizeof(struct DNode));
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
        return;
    }

    struct DNode* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
```

DATA SCIENCE

```
    }
    temp->next = newNode;
    newNode->prev = temp;
}
```

2. Deletion

- **Delete from Beginning**

```
void deleteBeginningDLL() {
    if (head == NULL) return;
    struct DNode* temp = head;
    head = head->next;
    if (head != NULL)
        head->prev = NULL;
    free(temp);
}
```

- **Delete from a Specific Position**

```
void deleteAtPositionDLL(int pos) {
    if (head == NULL) return;
    if (pos == 1) {
        struct DNode* temp = head;
        head = head->next;
        if (head != NULL)
            head->prev = NULL;
        free(temp);
        return;
    }

    struct DNode* temp = head;
    for (int i = 1; i < pos - 1 && temp->next != NULL; i++) {
        temp = temp->next;
    }
    if (temp->next == NULL) return;

    struct DNode* toDelete = temp->next;
    temp->next = temp->next->next;
    if (temp->next != NULL)
        temp->next->prev = temp;
    free(toDelete);
}
```

- **Delete from End**

```
void deleteEndDLL() {
    if (head == NULL) return;
    if (head->next == NULL) {
        free(head);
        head = NULL;
        return;
    }
```

DATA SCIENCE

```
    struct DNode* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->prev->next = NULL;
    free(temp);
}
```

3. Display

```
void displayListDLL() {
    struct DNode* temp = head;
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

Circular Singly Linked List

A **Circular Singly Linked List (CSLL)** is a variation of a singly linked list where the last node **does not point to NULL** but instead **points back to the first node**, forming a circular structure. This allows traversal from any node, and the list does not have a distinct "end" like a regular singly linked list.

Node Representation

```
struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL; // Global head pointer for Circular Singly Linked List
```

Program to implement Circular Single Linked List

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node *next;
};

// Global pointer to keep track of the head node
struct Node *head = NULL;

// Function to insert a node at the beginning
void insertAtBeginning(int value) {
```


DATA SCIENCE

```
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;

    if (head == NULL) {  // If list is empty
        newNode->next = newNode;
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = head;
        head = newNode;
    }
}

// Function to insert a node at the end
void insertAtEnd(int value) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;

    if (head == NULL) {  // If list is empty
        newNode->next = newNode;
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = head;
    }
}

// Function to insert a node at a specific position
void insertAtPosition(int value, int position) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = value;

    if (head == NULL || position == 1) {  // If list is empty or inserting at the start
        insertAtBeginning(value);
        return;
    }

    struct Node *temp = head;
    for (int i = 1; i < position - 1 && temp->next != head; i++) {
```

DATA SCIENCE

```
        temp = temp->next;
    }

    newNode->next = temp->next;
    temp->next = newNode;
}

// Function to delete a node from the beginning
void deleteFromBeginning() {
    if (head == NULL) {
        printf("List is empty!\n");
        return;
    }

    struct Node *temp = head, *last = head;

    while (last->next != head) {
        last = last->next;
    }

    if (head->next == head) {  // Only one node
        free(head);
        head = NULL;
    } else {
        head = head->next;
        last->next = head;
        free(temp);
    }
}

// Function to delete a node from the end
void deleteFromEnd() {
    if (head == NULL) {
        printf("List is empty!\n");
        return;
    }

    struct Node *temp = head, *prev = NULL;

    if (head->next == head) {  // Only one node
        free(head);
        head = NULL;
    } else {
        while (temp->next != head) {
            prev = temp;
            temp = temp->next;
        }
```

DATA SCIENCE

```
        prev->next = head;
        free(temp);
    }
}

// Function to delete a node from a specific position
void deleteFromPosition(int position) {
    if (head == NULL) {
        printf("List is empty!\n");
        return;
    }

    if (position == 1) {  // Delete first node
        deleteFromBeginning();
        return;
    }

    struct Node *temp = head, *prev = NULL;
    for (int i = 1; i < position && temp->next != head; i++) {
        prev = temp;
        temp = temp->next;
    }

    if (temp->next == head) {  // If last node
        deleteFromEnd();
    } else {
        prev->next = temp->next;
        free(temp);
    }
}

// Function to search for an element
void search(int key) {
    if (head == NULL) {
        printf("List is empty!\n");
        return;
    }

    struct Node *temp = head;
    int found = 0;

    do {
        if (temp->data == key) {
            printf("Element %d found in the list.\n", key);
            found = 1;
            break;
        }
```

DATA SCIENCE

```
        temp = temp->next;
    } while (temp != head);

    if (!found)
        printf("Element %d not found in the list.\n", key);
}

// Function to display the circular linked list
void display() {
    if (head == NULL) {
        printf("List is empty!\n");
        return;
    }

    struct Node *temp = head;
    printf("Circular Linked List: ");

    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);

    printf("(back to head)\n");
}

// Main function to test the operations
int main() {
    insertAtEnd(10);
    insertAtEnd(20);
    insertAtEnd(30);
    insertAtBeginning(5);

    display();  // Output: 5 -> 10 -> 20 -> 30 -> (back to head)

    insertAtPosition(15, 3);
    display();  // Output: 5 -> 10 -> 15 -> 20 -> 30 -> (back to head)

    deleteFromPosition(3);
    display();  // Output: 5 -> 10 -> 20 -> 30 -> (back to head)

    deleteFromBeginning();
    display();  // Output: 10 -> 20 -> 30 -> (back to head)

    deleteFromEnd();
    display();  // Output: 10 -> 20 -> (back to head)

    search(20); // Output: Element 20 found in the list.
```

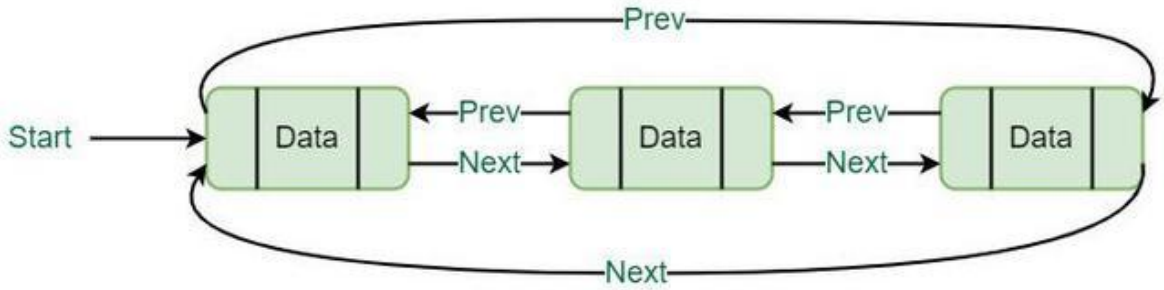
DATA SCIENCE

```
    search(50); // Output: Element 50 not found in the list.

    return 0;
}
```

Double Circular linked List

A circular doubly linked list is defined as a circular linked list in which each node has two links connecting it to the previous node and the next node.



Characteristics of Circular Doubly Linked List :

A circular doubly linked list has the following properties:

- **Circular**: A circular doubly linked list's main feature is that it is circular in design.
- **Doubly Linked**: Each node in a circular doubly linked list has two pointers – one pointing to the node before it and the other pointing to the node after it.
- **Header Node**: At the start of circular doubly linked lists, a header node or sentinel node is frequently used. This node is used to make the execution of certain operations on the list simpler even though it is not a component of the list's actual contents.

Applications of Circular Doubly Linked List :

Circular doubly linked lists are used in a variety of applications, some of which include:

- **Implementation of Circular Data Structures:** Circular doubly linked lists are extremely helpful in the construction of circular data structures like circular queues and circular buffers, which are both circular in nature.
- **Implementing Undo-Redo Operations**: Text editors and other software programs can use circular doubly linked lists to implement undo-redo operations.
- **Music Player Playlist**: Playlists in music players are frequently implemented using circular doubly linked lists. Each song is kept as a node in the list in this scenario, and the list can be circled to play the songs in the order they are listed.

- **Cache Memory Management**: To maintain track of the most recently used cache blocks, circular doubly linked lists are employed in cache memory management.

Program for Circular Double Linked List

Insertion Operations

1. Insert at the End

```
void insertAtEnd(int data) {
   Node* newNode = (Node*)malloc(sizeof(Node));
   if (newNode == NULL) {
      printf("Memory allocation failed.\n");
      return;
   }
   newNode->data = data;
   if (head == NULL) {
      head = newNode;
      newNode->next = newNode;
      newNode->prev = newNode;
   } else {
      Node* tail = head->prev; // Last node
      tail->next = newNode;
      newNode->prev = tail;
      newNode->next = head;
      head->prev = newNode;
   }
   printf("Inserted %d at the end.\n", data);
}
```

Explanation:

- If the list is empty, the new node points to itself, forming a circular list.
- If the list is not empty, we update the last node (tail) to point to the new node.
- The new node's next points to head, maintaining circularity.

Insert at the Beginning

```
void insertAtBeginning(int data) {
   Node* newNode = (Node*)malloc(sizeof(Node));
   if (newNode == NULL) {
      printf("Memory allocation failed.\n");
      return;
   }
   newNode->data = data;
```

DATA SCIENCE

```
    if (head == NULL) {
        head = newNode;
        newNode->next = newNode;
        newNode->prev = newNode;
    } else {
        Node* tail = head->prev;
        newNode->next = head;
        newNode->prev = tail;
        tail->next = newNode;
        head->prev = newNode;
        head = newNode;
    }
    printf("Inserted %d at the beginning.\n", data);
}
```

Explanation:

- If the list is empty, the new node becomes the only node, pointing to itself.
- If the list is not empty, the new node is inserted before the current head, and pointers are adjusted.

Insert at a Specific Position

```
void insertAtPosition(int data, int position) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }
    newNode->data = data;
    if (position == 1) {
        insertAtBeginning(data);
        return;
    }

    Node* temp = head;
    for (int i = 1; i < position - 1 && temp->next != head; i++) {
        temp = temp->next;
    }
    if (temp->next == head) {
        insertAtEnd(data);
        return;
    }
    newNode->next = temp->next;
    newNode->prev = temp;
    temp->next->prev = newNode;
    temp->next = newNode;
    printf("Inserted %d at position %d.\n", data, position);
}
```

DATA SCIENCE

Explanation:

- Traverse to the (position-1) node.
- Insert the new node between temp and temp->next.
- If position is greater than the list length, insert at the end.

Deletion Operations

1. **Delete a Specific Node by Value**

```
void deleteNode(int key) {
   if (head == NULL) {
      printf("List is empty.\n");
      return;
   }

   Node* curr = head;
   int found = 0;

   do {
      if (curr->data == key) {
         found = 1;
         break;
      }
      curr = curr->next;
   } while (curr != head);

   if (!found) {
      printf("Node with value %d not found.\n", key);
      return;
   }

   if (curr->next == curr && curr->prev == curr) {
      head = NULL;
   } else {
      curr->prev->next = curr->next;
      curr->next->prev = curr->prev;
      if (curr == head) {
         head = head->next;
      }
   }
   free(curr);
   printf("Deleted node with value %d.\n", key);
}
```

Explanation:

- Traverse to find the node with key.
- If found, update the previous and next node pointers, then free the node.

DATA SCIENCE

Delete at a Specific Position

```
void deleteAtPosition(int position) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    Node* temp = head;
    if (position == 1) {
        deleteNode(head->data);
        return;
    }

    for (int i = 1; i < position && temp->next != head; i++) {
        temp = temp->next;
    }

    if (temp->next == head) {
        printf("Position out of range.\n");
        return;
    }

    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;
    free(temp);
    printf("Deleted node at position %d.\n", position);
}
```

Explanation:

- Traverse to the position node.
- Update the previous and next node pointers.
- Free the node.

Traversal Operations

```
void traverseForward() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    Node* temp = head;
    printf("Forward Traversal: ");
    do {
        printf("%d ", temp->data);
        temp = temp->next;
```

DATA SCIENCE

```
    } while (temp != head);
    printf("\n");
}



void traverseBackward() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    Node* tail = head->prev;
    Node* temp = tail;
    printf("Backward Traversal: ");
    do {
        printf("%d ", temp->data);
        temp = temp->prev;
    } while (temp != tail);
    printf("\n");
}
```

Applications of linked list

1. Implementation of stacks and queues
2. Implementation of graphs: Adjacency list representation of graphs is the most popular which uses a linked list to store adjacent vertices.
3. Dynamic memory allocation: We use a linked list of free blocks.
4. Maintaining a directory of names
5. Performing arithmetic operations on long integers
6. Manipulation of polynomials by storing constants in the node of the linked list
7. Representing sparse matrices
8. Image viewer – Previous and next images are linked and can be accessed by the next and previous buttons.
9. Previous and next page in a web browser – We can access the previous and next URL searched in a web browser by pressing the back and next buttons since they are linked as a linked list.
10. Music Player – Songs in the music player are linked to the previous and next songs. So you can play songs either from starting or ending of the list.
11. GPS navigation systems- Linked lists can be used to store and manage a list of locations and routes, allowing users to easily navigate to their desired destination.

DATASCIENCE

12. Robotics- Linked lists can be used to implement control systems for robots, allowing them to navigate and interact with their environment.
13. Task Scheduling- Operating systems use linked lists to manage task scheduling, where each process waiting to be executed is represented as a node in the list.
14. Image Processing- Linked lists can be used to represent images, where each pixel is represented as a node in the list.
15. File Systems- File systems use linked lists to represent the hierarchical structure of directories, where each directory or file is represented as a node in the list.
16. Symbol Table- Compilers use linked lists to build a symbol table, which is a data structure that stores information about identifiers used in a program.
17. Undo/Redo Functionality- Many software applications implement undo/redo functionality using linked lists, where each action that can be undone is represented as a node in a doubly linked list.
18. Speech Recognition- Speech recognition software uses linked lists to represent the possible phonetic pronunciations of a word, where each possible pronunciation is represented as a node in the list.
19. Polynomial Representation- Polynomials can be represented using linked lists, where each term in the polynomial is represented as a node in the list.
20. Simulation of Physical Systems- Linked lists can be used to simulate physical systems, where each element in the list represents a discrete point in time and the state of the system at that time.
21. Useful for implementation of a queue. Unlike this implementation, we don't need to maintain two-pointers for the front and rear if we use a circular linked list. We can maintain a pointer to the last inserted node and the front can always be obtained as next of last.
22. Circular lists are useful in applications to go around the list repeatedly. For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application. It is convenient for the operating system to use a circular list so that when it reaches the end of the list it can cycle around to the front of the list.
23. Circular Doubly Linked Lists are used for the implementation of advanced data structures like the Fibonacci Heap.

DATA SCIENCE

24. Circular linked lists can be used to implement circular queues, which are often used in operating systems for scheduling processes and managing memory allocation.
25. Used in database systems to implement linked data structures, such as B+ trees, which are used to optimize the storage and retrieval of data.
26. Circular linked lists can be used in networking. For instance, to implement circular buffers for streaming data, such as video and audio, in networking applications.
27. Video games use circular linked lists to manage sprite animations. Each frame of the animation is represented as a node in the list, and the last frame is connected to the first frame to create a loop.
28. Circular linked lists can be used to represent a buffer of audio or signal data in signal processing applications. The last node is connected to the first node to create a loop, and the processing algorithms can efficiently iterate over the data.
29. Traffic light control systems use circular linked lists to manage the traffic light cycles. Each phase of the traffic light cycle is represented as a node in the list, and the last node is connected to the first node to create a loop.

DATA SCIENCE